

Esta apresentação foi criada e adaptada pelos professores Diogo Cezar Teixeira Batista e Francisco Pereira Junior (Thesko).

Laboratório de Informática

Prof. Francisco Pereira Junior (Thesko)

Universidade Tecnológica Federal do Paraná | Engenharia de Software / TADS
Laboratório de Informática | 2020/01 | versão-1.0



MINISTÉRIO DA
EDUCAÇÃO



Linguagem C

struct
registros

Prof. Francisco Pereira Junior (Thesko)



MINISTÉRIO DA
EDUCAÇÃO



REGISTROS

- Tipos de dados primitivos .VS. estruturados
 - Primitivos:
reais (float, double), inteiros (int), caractere (char)
 - Estruturados:
informações compostas por diversos campos, possivelmente com tipos de dados diferentes, para se construir um **novo** tipo

REGISTROS

- **o que são?** formas de criar o seu próprio tipo de dados composto (**novo tipo**)
- um registro é um “pacote” de variáveis
 - possivelmente de tipos diferentes
 - agrupadas sob um único nome

REGISTROS

Pessoa	
Tipo de dados	Campo ou Membro
String	nome
String	endereço
Inteiro	idade
String	data de nascimento
Real	peso
Real	altura

- tipo de dados composto
- tipo de dados estruturado
- tipo de dado definido pelo usuário
- registro
- struct
- variável heterogênea

Coleção de variáveis relacionadas!

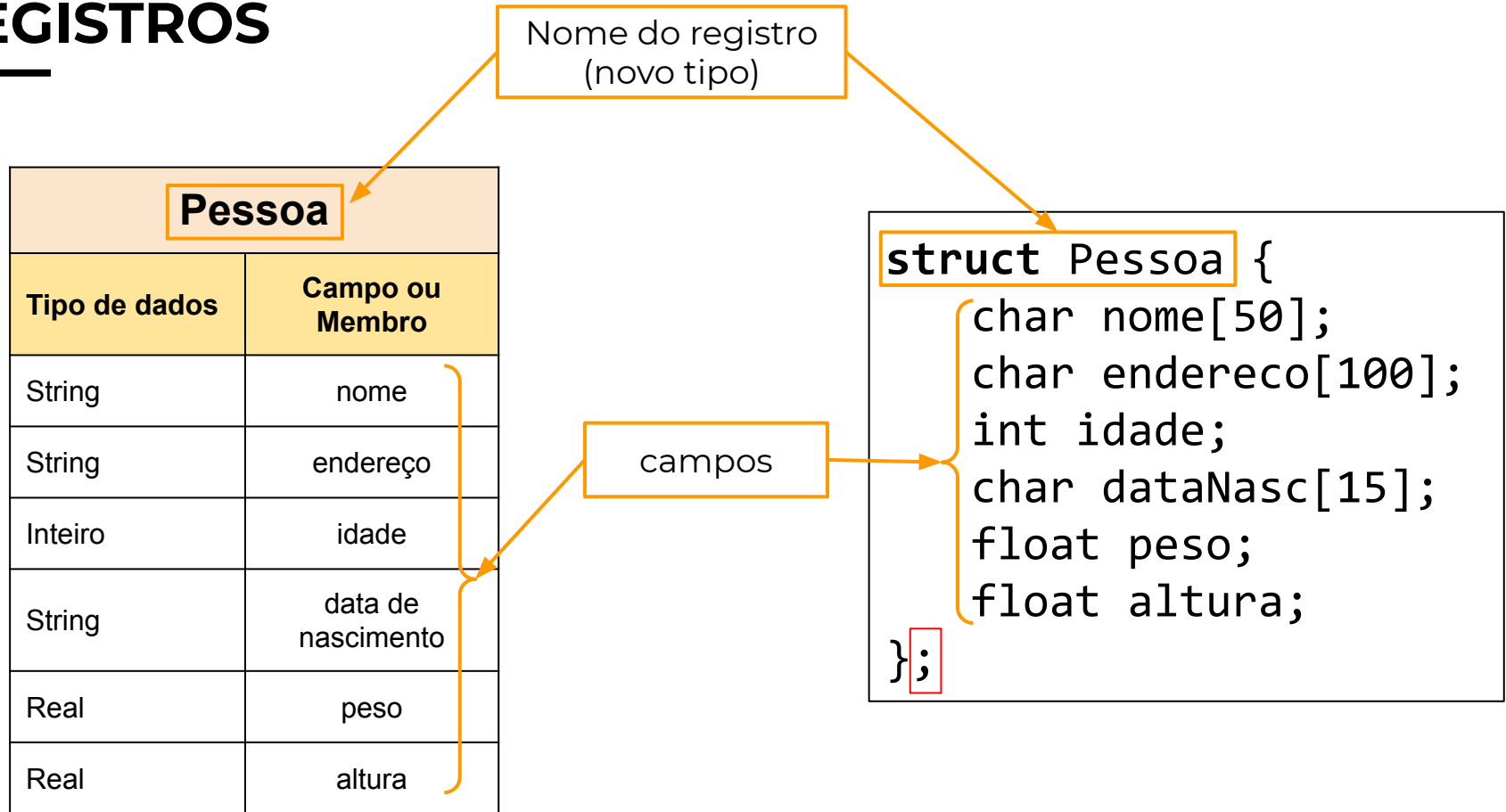
REGISTROS

SINTAXE BÁSICA

```
struct nome_do_registro {  
    tipo_de_dado nome_campo1;  
    tipo_de_dado nome_campo2;  
    ...  
};
```

- palavra chave **struct** → define uma “etiqueta” que dá nome à definição da estrutura (seu **novo** tipo);
- esta definição também é um comando, então deve terminar por ponto-e-vírgula;
- os campos são declarados entre bloco (chaves);

REGISTROS



REGISTROS

- **regra** → todos os campos **de um mesmo registro** devem possuir nomes diferentes;
 - campos de **registros diferentes** podem ter o mesmo nome;
- os campos podem ser de qualquer tipo, inclusive de um outro **struct**;
 - quando isso ocorre chamamos de **estrutura aninhada**;

REGISTROS

- a definição de uma estrutura *não reserva espaço na memória*;
- *o que faz então?* → cria um *novo tipo de dados*!
- pode ser utilizado para declarar variáveis;
- assim como qualquer tipo pode ser declarado como *vetor* ou *matriz*;

CONSTRUÇÃO DE REGISTROS

os registros podem ser construídos de 2 formas:

DEFINE A ESTRUTURA E DEPOIS AS VARIÁVEIS

```
struct Aluno {  
    char nome[50];  
    int ra;  
    int idade;  
};  
...  
struct Aluno aluno1;  
struct Aluno aluno2;
```

DEFINE AS VARIÁVEIS JUNTO COM A ESTRUTURA

```
struct Aluno{  
    char nome[50];  
    int ra;  
    int idade;  
} aluno1, aluno2;
```

ACESSANDO OS CAMPOS DE UM REGISTRO

- podemos acessar **individualmente** os campos de um registro como se fossem variáveis comuns;
- sintaxe para manipular os campos dos registros:

`<nome_da_variável>.<nome_do_campo>`

```
struct Aluno {  
    char nome[50];  
    int ra;  
    int idade;  
};  
  
struct Aluno aluno;
```

SINTAXE ACESSO AOS CAMPOS

```
printf("Digite o nome do aluno: ");  
gets(aluno.nome);  
printf("Digite o código do aluno: ");  
scanf("%d", &aluno.ra);  
printf("Digite a idade do aluno: ");  
scanf("%d", &aluno.idade);
```

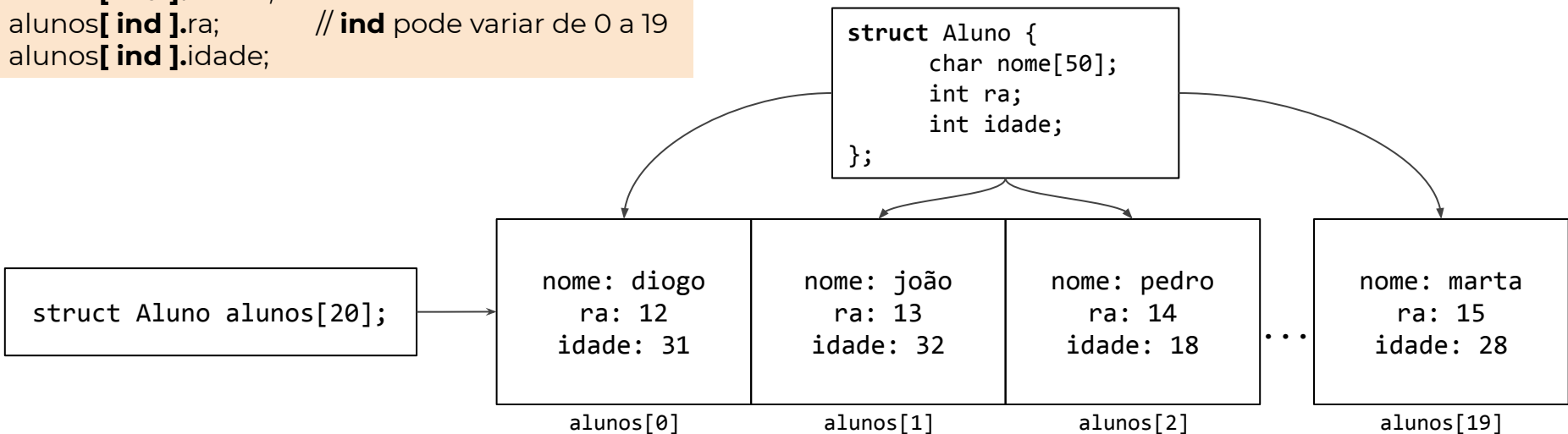
REGISTROS

- (APLICAÇÃO) como armazenar **nome**, **ra** e **idade** de 20 alunos de uma turma?
 - com variáveis? 60 variáveis diferentes (como fica a relação?)
 - com vetores? um vetor para os nomes, um para os ras e outro para as idades (como fica a relação?)
 - com registros? em um único vetor de 20 elementos

REGISTROS EM VETORES E MATRIZES

- cada posição do vetor representa um **Aluno** com **nome**, **ra** e **idade**

```
alunos[ ind ].nome;  
alunos[ ind ].ra;      // ind pode variar de 0 a 19  
alunos[ ind ].idade;
```



PASSANDO ESTRUTURAS PARA FUNÇÕES

- você pode enviar uma estrutura inteira para uma função

EXEMPLO DE ESTRUTURAS PARA FUNÇÕES

```
struct Aluno {  
    char nome[50];  
    int ra;  
    int idade;  
};  
  
void func1(struct Aluno param);  
...
```

EXEMPLO DA CHAMADA

```
int main() {  
    struct Aluno aluno;  
    ...  
    printf("Digite o nome do aluno: ");  
    gets(aluno.nome);  
    printf("Digite o código do aluno: ");  
    scanf("%d", &aluno.ra);  
    printf("Digite a idade do aluno: ");  
    scanf("%d", &aluno.idade);  
    ...  
    ...  
    func1( aluno );  
    ...  
    ...  
    return 0;  
}
```

PASSANDO ESTRUTURAS PARA FUNÇÕES

- você pode enviar (individualmente) campos/membros de uma estrutura para uma função

EXEMPLO DE ESTRUTURAS PARA FUNÇÕES

```
struct Aluno {  
    char nome[50];  
    int ra;  
    int idade;  
};  
  
void func2(int value);  
...
```

EXEMPLO DA CHAMADA

```
int main() {  
    struct Aluno aluno;  
    ...  
    printf("Digite o nome do aluno: ");  
    gets(aluno.nome);  
    printf("Digite o código do aluno: ");  
    scanf("%d", &aluno.ra);  
    printf("Digite a idade do aluno: ");  
    scanf("%d", &aluno.idade);  
    ...  
    ...  
    func2( aluno.ra );  
    ...  
    ...  
    return 0;  
}
```

DEFININDO TIPOS

- na linguagem C podemos redefinir **tipos de dados**;
- utiliza-se a palavra reservada `typedef`;
- é comum o uso de `typedef` em estruturas de dados;
 - evita-se o uso da palavra chave `struct` a cada declaração de variável;
- basicamente criamos um apelido para um tipo de dados;

UTILIZAÇÃO DO TYPEDEF

```
typedef int meuInteiro;  
meuInteiro a;  
meuInteiro b[10];
```

DEFININDO TIPOS

UTILIZANDO TYPEDEF

```
typedef struct Aluno {  
    char nome[50];  
    int ra;  
    int idade;  
} tpAluno;
```

UTILIZAÇÃO

```
// SEM TYPEDEF  
struct Aluno aluno;  
  
// COM TYPEDEF  
tpAluno aluno;
```

COMPOSIÇÃO

- um registro pode ter como campo (membro) um outro registro
- um registro pode ter como campo (membro) um vetor, inclusive de outro registro

EXEMPLO COMPOSIÇÃO (sem typedef)

```
struct Aluno {  
    char nome[50];  
    int idade;  
};  
  
struct Escola {  
    char nome[50];  
    struct Aluno aluno[100];  
};  
...  
struct Escola teste;
```

EXEMPLO COMPOSIÇÃO (com typedef)

```
typedef struct Aluno {  
    char nome[50];  
    int idade;  
} tpAluno;  
  
typedef struct Escola {  
    char nome[50];  
    tpAluno aluno[100];  
} tpEscola;  
...  
tpEscola teste;
```

COMPOSIÇÃO

- para acessar campos/membros de uma estrutura composta

<nome_da_struct_externa>.<nome_da_struct_interna>.<nome_do_campo>

```
struct Aluno {  
    char nome[50];  
    int idade;  
};  
  
struct Escola {  
    char nome[50];  
    struct Aluno aluno[100];  
};  
...  
struct Escola teste;
```

SINTAXE ACESSO AOS CAMPOS

```
gets(teste.nome);  
gets(teste.aluno[i].nome);  
scanf("%d", &teste.aluno[i].idade);
```

Esta apresentação foi criada e adaptada pelos professores Diogo Cezar Teixeira Batista e Francisco Pereira Junior (Thesko).

Laboratório de Informática

Prof. Francisco Pereira Junior (Thesko)

Universidade Tecnológica Federal do Paraná | Engenharia de Software / TADS
Laboratório de Informática | 2020/01 | versão-1.0

